DAY1

Task 1 : FROG

SOLUTION
A naïve $O(N^3)$ time algorithm for the problem iterates through all $O(N^2)$ line segments induced by the point set *S*, and determines how far each segment spacing can be extended to either direction within the point set (*O(# landings) = O(N)*).

An efficient $O(N^2)$ time algorithm for the problem is based on an algorithm for finding an *equally-spaced collinear subset of a set.* The algorithm works by "overlapping" all equally spaced triples in order to determine all maximal equally-spaced collinear subsets. The "overlapping" is performed by constructing an undirected graph where for each equally-spaced triple (p_A, p_B, p_C) we create nodes <*A,B*> and <*B,C*> and the edge (<*A,B*>, <*B,C*>); connected components in this graph correspond to maximal equally-spaced collinear subsets in the original set. Observe that a frog path is simply a linear chain of connected nodes (with at least one edge and two nodes, meaning at least 3 flattened plants) in this graph. Each node in this graph has degree at most two, so the edge set and vertex set both have size $O(N^2)$. Hence we can find all maximal equally-space collinear subsets in $O(N^2)$ time from the graph.

The only detail here is how to efficiently find the equally-spaced triples from which the graph is created. The obvious method of iterating over all triples of flattened plants would worsen the complexity to $O(N^3)$. If instead the field is stored as a two-dimensional array (every plant has an entry) giving the identity of the landing on that plant (e.g., if the 100th flattened plant were at (10,12), then the array value at (10,12) is 100), you can loop over pairs of flattened plants p_A and p_B, and then look up p_C from the array in constant time since you know what the location of p_C must be if it exists. This strategy takes $O(N^2)$ time but uses O(*field size*) memory – in particular, it needs 5000*5000 entries of a short integer each, or 50MB. Because the above graph also needs $O(N^2)$ space to store it, this strategy unfortunately would exceed the memory limit of 64MB. However, as this array is very sparse, it may be stored in memory as a hash table, which in the expected case does not affect the time complexity (but which in the worst case does). The third and best option is to construct the graph in linear time and constant memory by sorting the locations (e.g., row major, column minor) and keeping 3 pointers into the list (*A*, *B*, and *C* for pointing to p_A, p_B, and p_C, $A<B<C$) as follows; loop *A* over all values, and for each *A* march *B* and *C* down the list, moving either *B* or *C* forward at each step so as to try to maintain as close to equal spacing as possible; when exactly equal spacing is found, enter the nodes and edge into the graph.

There is also an $O(N^2)$ dynamic programming algorithm to solve this problem, which is plagued by the same memory problems as illustrated above. In addition to storing the identity matrix described above, store another $O(N^2)$ matrix containing whose rows are indexed by p_A; along the row are *N* entries, one per plant p_B giving the number of landings in a candidate frog path which goes through p_A and p_A but which only uses points which sort before p_A in the ordered list (i.e., pretend the field ends at p_A, and look for frog paths of any length in that smaller field – the idea is to find partial frog paths which violate none of the frog path conditions in the region of the field already examined). Assuming the table is filled up to row *A*, row *A+1* is filled by considering all $O(N)$ flattened plants *B* before p_A,

and if there is a flattened plant *C* such that *A*, *B*, and *C* are equally spaced, look up in the array the number of landings in the candidate frog path through *B* from *C*, increment by 1, and store as the *B*th entry in the row for *A*. If *C* would be outside the field, then enter it as having 2 flattenings. At the same time check to see if the next flattened plant (*D*) would be outside the graph, and if so, you have a completed frog path. To efficiently determine *C*, the same 50MB array as above is needed; a hash table can again be used, with no increase in average-case time complexity, but an increase in worst-case time complexity.

TESTING

The test data contains 25 test cases. Most of data are initially generated by random function, then they are modified by manual work.

Each test case has size *N* (the number of points) in the range between 10 and 5000. Among 25 test cases, 10 test cases have size $N \leq 1000$. The remaining 15 test cases have size $N \geq 2000$. The detail on the test data is summarized in the following table.

Each test case is worth 4 points. A program which implements a cubic time algorithm can solve the test cases within time limit such that their size $N \leq 1000$. An implementation of this algorithm should be able to get the first 10 test cases correct but will likely run out of time on all other cases (scoring 40% of the points).

Testing Data Description FROG

No	points, (r*c)	Description	Solution
1	18, (6 * 7)	Sample data in task	4
2	10, (10 * 10)	Manually designed	5
3	25, (50 * 50)	Manually designed	13
4	50, (10 * 10)	Several Lines + random points	10
5	100, (20 * 20)	modified random point set	10
6	300, (30 * 30)	modified random point set	15
7	500, (55 * 55)	Several Lines + random points	28
8	500, (100 * 100)	Special case for no solution	0
9	1000, (100 * 100)	Several Lines + random points	34
10	1000, (1000 * 1000)	Several Lines + random points	250
11	2000, (50 * 50)	Random (uniform) points	25
12	2000, (100 * 200)	Several Lines + random points	33
13	2000, (1000 * 2000)	Several Lines + random points	333
14	3000, (60 * 60)	Uniformly random points	31
15	3000, (500 * 500)	X shapes and random points	500
16	3000, (5000 * 1)	Horizontal line	20
17	3000, (5 * 1000)	Several Lines + random points	17
18	4000, (100 * 100)	Random points (uniformly)	34
19	4000, (200 * 20)	Very dense points set	200
20	4000, (1000 * 1000)	Several Lines + random points	500
21	4000, (5000 * 5000)	Several Lines + random points	311
22	5000, (100 * 100)	Chess board style	100

23	5000, (1000 * 1000)	Several Lines + random points	334
24	5000, (3000 * 3000)	Irregular linear points	1000
25	5000, (5000 * 5000)	Modified random points	72

BACKGROUND

The problem "The Troublesome Frog" is related to the problem for detecting spatial regularity in images. Spatial regularity detection is an important problem in a number of domains such as computer vision, scene analysis, and landmine detection from infrared terrain images. The AMESCS(All Maximum Equally-Spaced Collinear Subset) problem is defined as follows. Given a set P of n points in E^d, find all maximal equally-spaced, collinear subset of points. Kahng and Robins[1] present an optimal quadratic time algorithm for solving the AMESCS problem.

Reference

[1] A B. Kahng and G. Robins, **Optimal algorithms extracting spatial regularity in images**, Pattern Recognition Letters, 12, 757-764, 1991.

UTOPIA DIVIDED

Solution

The two dimensional problem can be solved by two separate one dimensional subproblems. The one dimensional problem is: Given *N* code numbers and a sequence of *N* region signs (each of which is + or -), produce a sequence of *N* signed code values $\{x_i\}$ so that the sign of $\Sigma_{i<k}\, x_i$ matches the i^{th} region sign.

We do this by first sorting the *N* input code numbers into increasing order, and then assigning alternating signs to them so that $|x_i| > |x_{i+1}|$, though $x_i > 0$ iff $x_{i+1} < 0$. The key observation is that if we have used a segment $(x_k, \ldots, x_{m)}$ of the data, then $0 \le \left|\Sigma_{k\le i\le m}\, x_i\right| \le \left|x_m\right|$, and the sign of the sum matches that of x_m. If the next code is to reverse the sign, we simply use x_{m+1}, which guarantees the invariant of the last sentence still holds. A similar argument guarantees that using x_{k-1} retains the sign of the sum.

The next important issue is where to start. Recall that keeping the sign of the sum the same requires taking a new value from the right. So count the number of sign changes in the input sequence of region signs. If this value is c, give x_c the first input sign, thus determining the alternation. The output can now be produced in a fairly straightforward manner.

Hence there is an O(*N* lg *N*) solution that can be coded (if not discovered) quite easily. It is also quite reasonable to sole the problem by backtracking. This leads to an acceptable solution on smaller test cases (up to 8, or with care to 9 or even 10).

No	Size, N	Description
1	N = 4	Example 2
2	N = 10	Key value : 1~20: Circular plane sweep
3	N = 10	Key value : Starting from 20, step 1 or 2, scrambled
4	N = 30	Key value : Starting from 30, step 1 or 2, scrambled
5	N = 30	Key value : Starting from 200, step 1 to 3, scrambled
6	N = 50	Key value : Starting from 1, step 1 to 3, scrambled
7	N = 50	Key value : Starting from 150, step 1 to 3, scrambled
8	N = 50	Key value : Starting from 300, step 1 to 4, scrambled Visit plane 1 except last visit. On last, visit plane 3
9	N = 100	Key value : Starting from 1000, step 1 to 4, scrambled. Circular plane visit
10	N = 500	Key value : 1, 3, 5, …, 1999. Circular plane visit
11	N = 700	Key value : Starting from 1, step 1 to 5, scrambled. Visit plane 1, 2 only
12	N = 1000	Key value : Starting from 4000, step 1 to 5, scrambled
13	N = 1500	Key value : Starting from 9000, step 1 to 5, scrambled
14	N = 2000	Key value : Starting from 15000, step 1 to 5, scrambled
15	N = 2500	Key value : Starting from 30000, step 1 to 5, scrambled
16	N = 3000	Key value : Starting from 1, step 1 to 7, scrambled. Visit plane 1, 4 only
17	N = 3500	Key value : Starting from 20000, step 1 to 7, scrambled
18	N = 4000	Key value : Starting from 30000, step 1 to 7, scrambled
19	N = 4500	Key value : Starting from 50000, step 1 to 7, scrambled
20	N = 5000	Key value : Starting from 60000, step 1 to 7, scrambled
21	N = 6000	Key value : Starting from 1, step 1 to 9, scrambled. Visit plane 2, 4 only
22	N = 7000	Key value : Starting from 40000, step 1 to 7, scrambled
23	N = 8000	Key value : Starting from 50000, step 1 to 5, scrambled
24	N = 9000	Key value : Starting from 30000, step 1 to 6, scrambled
25	N = 10000	Key value : Starting from 80000~99999, scrambled

XOR

Solution

In addition to pixels, we examine *grid points*, which are intersection points of two grid lines. An *image corner* is grid point adjacent to an odd number of black pixels. This way, an image is all white if and only if there are no image corners.

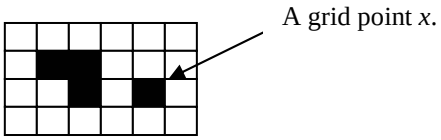


Figure 1. An example image.

For example, in Figure 1 the grid point *x* is adjacent to four pixels, of which 1 is black and 3 are white. This way, *x* is also an image corner. In total, there are 10 image corners in the image in Figure 1.

We may now try to find a solution by working from the input image backward to an all white image by doing XOR operations. It can be shown that if there are black pixels in the image, then it is always possible to find at least three such image corners that they are corners of the same rectangle. Now we use the following algorithm.

Algorithm 1. Two-optimal XOR

1. If there are black pixels in the image, find three such image corners that they are corners of the same rectangle, and use the respective XOR call. Go to 1.
2. Halt.

It can be shown that repeating the execution of Step 1 will eventually remove all black pixels and, in this way, create a solution. It can also be shown that we can reduce the number of image corners by at most four with a single XOR call, and the above algorithm always reduces the number of image corners by at least two. Therefore, the above algorithm always uses at most twice as many XOR calls than the best possible solution (and we say that Algorithm 1 is *2-optimal*).

We can produce variations of Algorithm 1 by different ways of choosing the image corners, and they may produce different numbers of XOR calls.

There are also other approaches to solve this problem. As an extremely naïve solution, one may do one XOR call for each black pixel in the image. Then, you will do as many XOR calls as there are black pixels in the image. As an improvement, you may also try to identify all-black rectangles, which may considerably reduce the number of XOR calls from the previous naïve solution.

BATCH SCHEDULING

A. Solution

This problem can be solved using dynamic programming. Let C_i be the minimum total cost of all partitionings of jobs $J_i, J_{i+1}, \ldots, J_n$ into batches. Let $C_i(k)$ be the minimum total cost when the first batch is selected as $\{J_i, J_{i+1}, \ldots, J_{k-1}\}$. That is, $C_i(k) = C_k + (S + T_i + T_{i+1} + \ldots + T_{k-1}) * (F_i + F_{i+1} + \ldots + F_n)$.

Then we have that
$C_i = \min \{ C_i(k) \mid k = i+1, \ldots, n+1\}$ for $1 \le i \le n$,
and $C_{n+1} = 0$.

(a) O(n^2) Time Algorithm

The time complexity of the above algorithm is $O(n^2)$.

(b) O(n) Time Algorithm

Investigating the property of $C_i(k)$, P. Bucker[1] showed that this problem can be solved in $O(n)$ time.

From $C_i(k) = C_k + (S + T_i + T_{i+1} + \ldots + T_{k-1}) * (F_i + F_{i+1} + \ldots + F_n)$, we have that for $i < k < l$,

$$C_i(k) \le C_i(l) \Leftrightarrow C_l - C_k + (T_k + T_{k+1} + \ldots + T_{l-1}) * (F_i + F_{i+1} + \ldots + F_n) \ge 0$$
$$\Leftrightarrow (C_k - C_l) / (T_k + T_{k+1} + \ldots + T_{l-1}) \le (F_i + F_{i+1} + \ldots + F_n)$$

Let $g(k,l) = (C_k - C_l) / (T_k + T_{k+1} + \ldots + T_{l-1})$ and $f(i) = (F_i + F_{i+1} + \ldots + O_n)$

Property 1: Assume that $g(k,l) \le f(i)$ for $1 \le i < k < l$. Then $C_i(k) \le C_i(l)$
Property 2: Assume $g(j,k) \le g(k,l)$ for some $1 \le j < k < l \le$ n. Then for each i with $1 \le i < j$, $C_i(j) \le C_i(k)$ or $C_i(l) \le C_i(k)$.

Property 2 implies that if $g(j,k) \le g(k,l)$ for $j < k < l$, C_k is not needed for computing F_i. Using this property, a linear time algorithm can be designed, which is given in the following.

Algorithm Batch

The algorithm calculates the values C_i for $i = n$ down to 1. It uses a queue-like list $Q = (i_r, i_{r-1}, \ldots, i_2, i_1)$ with tail i_r and head i_1 satisfying the following properties:

$i_r < i_{r-1} < \ldots < i_2 < i_1$ and
$g(i_r, i_{r-1}) > g(i_{r-1}, i_{r-2}) > \ldots > g(i_2, i_1)$ -------- (1)

When C_i is calculated,
1. // Using $f(i)$, remove unnecessary element at head of Q.
If $f(i) \ge g(i_2,i_1)$, delete i_1 from Q since for all $h \le i$, $f(h) \ge f(i) \ge g(i_2,i_1)$ and $C_h(i_2) \le C_h(i_1)$ by Property 1.

Continue this procedure until for some $t \geq 1$, $g(i_r, i_{r-1}) > g(i_{r-1}, i_{r-2}) > \ldots > g(i_{t+1}, i_t) > f(i)$.

Then by Property 1, $C_i(i_{v+1}) > C_i(i_v)$ for $v = t, \ldots, r-1$ or
$r = t$ and Q contains only i_t.
Therefore, $C_i(i_t)$ is equal to $\min\{C_i(k) \mid k = i+1, \ldots, n+1\}$.

2. // When inserting i at the tail of Q, maintain Q for the condition (1) to be satisfied.
If $g(i, i_r) \leq g(i_r, i_{r-1})$, delete i_r from Q by Property 2.
Continue this procedure until $g(i, i_v) > g(i_v, i_{v-1})$.
Append i as a new tail of Q.

Analysis

Each i is inserted into Q and deleted from Q at most once. In each insertion and deletion, it takes a constant time. Therefore the time complexity is $O(n)$.

B. Test Data Information and Grading

In total, 20 test cases are prepared and tested. Each test case is of 5 credits. Among them, 19 test cases are randomly generated so that overflow does not occur during computing Fi. The remaining 1 test case is that setup time and all processing times and cost factors are 1.

The test cases are mainly prepared to distinguish whether the competitors design an efficient algorithm or not. Among 20 test cases, algorithm by enumeration may solve for three ones within the given time limit, and an $O(n^2)$ time algorithm may solve for 14 test cases within the given time limit. If the competitors submit a correct $O(n)$ time algorithm, they will get 100 credits.

BUS TERMINALS

Solution

The solution is based on the algorithm, running in $O(n^3)$ time, presented in [1]. Recently, this algorithm is slightly improved in [2], but its implementation is too complicated to accept it for the competition, so we use the algorithm in [1] as a solution.

The *diameter* of a bus network is the longest length of the route between any two bus stops in the bus network. Our goal is to find the minimum value of the diameters over all possible choices of the hubs and assignments of bus stops. As did in [1], we consider two cases separately. For it, we need some notations. Let D_1 be the minimum value of the longest length between two bus stops which are connected through only one hub over all possible choice of one hub, and let D_2 be the minimum value of the longest length between two bus stops which are connected through both two hubs over all possible choice of two hubs and the corresponding assignments of bus stops. The diameter can be found in the following way presented in [1]. First, compute D_1 and D_2. Next, output the minimum of D_1 and D_2 as the minimum diameter of the entire network.

First we will explain the computation of D_1. If a point p will be served as the hub through which the longest route passes, the longest length is $d(p, q) + d(p, r)$, where the points q and r is the farthest and the second farthest ones from p, respectively. Then $D_1 = \min_p \{ d(p, q) + d(p, r) \}$ over all points p of the input. This can be obtained in $O(n^2)$ time because the farthest and second farthest bus stops for each point p are easily found in $O(n)$ time. Second we will explain how to compute D_2 with a simple example. Note that in this case the longest route between two bus stops will pass both two hubs H_1 and H_2.

We consider all pairs of bus stops of the input as possible two hubs H_1 and H_2, and select the pair of the bus stops that gives a minimum diameter. Let at the beginning D_2 be sufficiently large (e.g., `maxint`). Consider now fixed two hubs H_1 and H_2. Each of the remaining $n - 2$ points will be initially connected to one of two hubs, say H_1. Sort the remaining $n - 2$ points in the array P in non-decreasing order according to the distance from the hub H_1 (Figure 1).

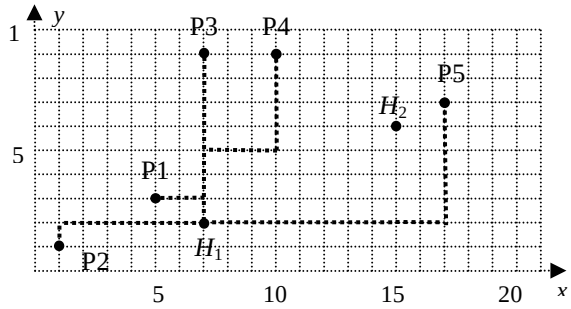


Figure 1

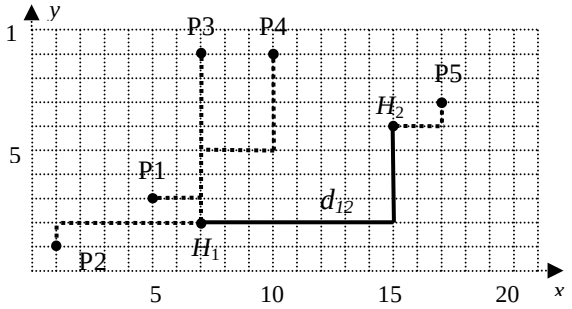


Figure 2

Denote by $r_1 = d(H_1, P[n\text{-}3])$, $r_2 = d(H_2, P[n\text{-}2])$ and $d_{12} = d(H_1, H_2)$. If $r_1 + d_{12} + r_2 < D_2$, then the point $P[n\text{-}2]$ is connected to the hub H_2 and set D_2 to the new value $D_2 = r_1 + d_{12} + r_2$. Figure 2 represents this step, $r_1 = d(H_1, P[n\text{-}3]) = d(H_1, P[4]) = 10$, $r_2 = d(H_2, P[n\text{-}2]) = (H_2, P[5]) = 3$, $d_{12} = d(H_1, H_2) = 12$, so $D_2 = r_1 + d_{12} + r_2 = 10 + 12 + 3 = 25$. Now we repeat the same procedure with $r_1 = d(H_1, P[n\text{-}4])$, $r_2 = d(H_2, P[n\text{-}3])$, same $d_{12} = d(H_1, H_2)$, and get $r_1 + d_{12} + r_2 = d(H_1, P[3]) + d_{12} + d(H_2, P[4]) = 7 + 12 + 8 = 27$. Since we got the new distance which is greater than the previous diameter, the value D_2 remains unchanged, so D_2 still has value 25. (If 25 is turned out to be the minimum of D_2 at the end of the procedure, the point $P[4]$ shall be connected to H_1 although its distance to H_2 is smaller than the distance from H_1.) This situation is represented in Figure 3 where the point $P[4]$ is connected with a thin line to H_2 which is shorter than the distance from H_1 to $P[4]$.

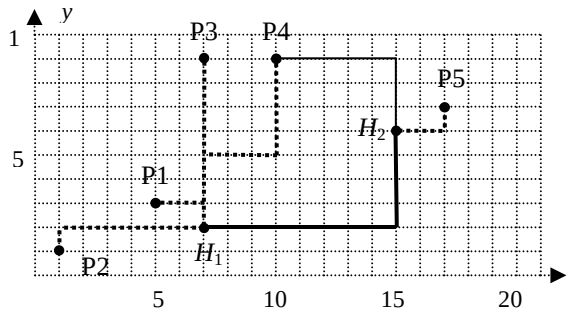


Figure 3

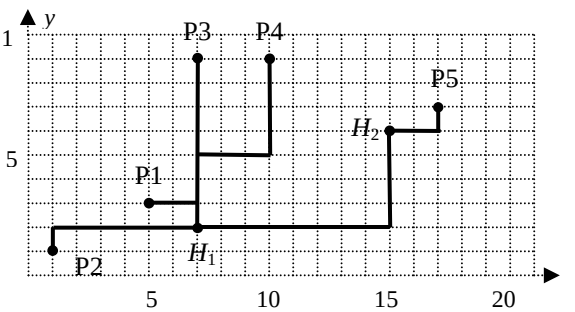


Figure 4

This procedure is repeated by decreasing the index of the array P one by one until the index 1 is reached. For the example, the minimum value of D_2 is 25 after the procedure and the corresponding network is shown in Figure 4.

Other approaches

Many contestants may take a (seemingly natural and intuitive) heuristic approach to connect each of $n - 2$ bus stops to the nearest one of two hubs. But this is wrong because there is a counter example. Of course, this approach can produce correct answers for some inputs (in six test cases with bold faces in the table).

We can make a *bruteforce* algorithm running in $O(n^4)$ time. It considers all pairs of points as hubs H_1 and H_2, and computes $D(H_1, H_2)$ for each pair in $O(n^2)$ time. But this approach will not produce the answer for large inputs within the time limit. Six tests are small enough that bruteforce approach will work.

Test Data Information

No.	n	size	Generation	Answer	Heuristic's Answer
1	2	10	Extreme case	**18**	**18**
2	7	20	Example 2	25	26
3	10	30	Random	**42**	**42**
4	10	30	Random	52	53
5	50	100	Random	167	168
6	100	20	Random	35	36
7	170	1000	Random	1884	1896
8	180	1000	Random	1845	1849
9	300	650	Dumbbell,-45-degree	**911**	**911**
10	300	650	Dumbbell,45-degree	**995**	**995**
11	400	675	Dumbbell,0-degree	**689**	**689**
12	400	20	20x20 grid	**39**	**39**
13	300	100	Random	186	188
14	300	1000	Random	1876	1882
15	350	150	Random	286	287
16	350	500	Random	945	946
17	350	2000	Random	3697	3709
18	400	1000	Random	1908	1912
19	400	5000	Random	9381	9405
20	500	500	Random	970	971

References

[1] J.-M. Ho, D. T. Lee, C.-H. Chang, C. K Wong**, Minimum diameter spanning trees and related problems,** *SIAM J. on Computing*, 20(5):987—997, 1991.

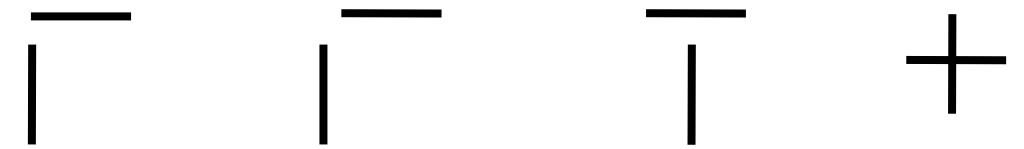
[2] T. Chan**, Semi-online maintenance of geometric optima and measures,** *13th ACM-SODA*, 474—483, 2002.

Handout for Two Rods

SOLUTION

The fastest approach we know is to perform six binary searches.

- Using entire rows/columns as the query rectangle, the top and bottom rows, and leftmost and rightmost columns containing any portion of a rod can be found using 4 binary searches, or $4\lceil \lg N \rceil$ calls to `rect`.
- We now have the smallest rectangle containing all of both rods. By checking the corners of this rectangle (4 calls to `rect`, each with a 1 by 1 query rectangle), we can determine the general form of the structure, as in the examples of the figure and their rotations.
- Finally the solution can be found in one or two more binary searches depending on the case.



3 corners intersect rods

2 opposite corners intersect rods

2 adjacent corners intersect rods

no corners intersect rods

This leads to a $6\lceil \lg N \rceil + 4$ solution. The maximum value of $\lceil \lg N \rceil$ in the test data is 14, so we have a solution that takes at most 88 calls to `rect`. With care, this can be reduced to $6\lceil \lg N \rceil + 1$. Notice that the queries we ask of the data have only two possible answers. As there are $N^4(N\text{-}1)^2/4$ possible placements of the rods, $\lceil \lg (N^4(N\text{-}1)^2) \rceil - 2 \approx \lceil 6 \lg N \rceil - 2$ calls are necessary, on average, for any algorithm. We do not claim our testing is exhaustive, so we simply take this as a worst case lower bound. We implemented two versions of the general approach suggested.

There are a number of variations on this approach. For example, one could try to finding the bounding rectangle more quickly when it is large. Approaches of this type tend to double the number of calls to `rect` and will lead to full marks on the 4 small cases and 3 marks on each of the larger cases.

The most naïve approach involves scanning individual cells to find some portion of a rod, then looking around for the rest of the rod. This can clearly lead to an N^2 solution, or even slightly worse if one is careless. The approach receives full marks in the four cases of size 10. Another exhaustive approach involves taking entire rows (or columns) as query rectangles to find the bounding rectangle, then applying a similar approach to find the rods inside this rectangle. This will require O(N) calls, though the constant will vary with details of the implementation. Depending on the details of the implementation, such an approach could also gain credit in several of the larger examples in which the rods are small and near a corner.

Testing Data Description for RODS

No	Grid Size, N	Solution 1	Solution 2	$6\lceil \lg N \rceil+4$
1	10	14	15	28
2	1000	51	51	64
3	5000	77	77	82
4	7000	75	76	82
5	10000	80	79	88
6	10	17	18	28
7	1000	50	51	64
8	5000	69	68	82
9	7000	77	76	82
10	10000	79	77	89
11	10	16	15	28
12	1000	62	63	64
13	5000	73	71	82
14	7000	79	76	82
15	10000	79	77	88
16	10	16	15	28
17	1000	52	51	64
18	5000	56	61	82
19	7000	65	64	82
20	10000	70	70	88